

Polimorfismo em Java

Prof. Marlos de Mendonça Corrêa

Apresentação

O polimorfismo e as interfaces em Java serão os pontos principais deste conteúdo. O polimorfismo é um dos principais conceitos quando se fala em programação OO e a interface é uma ferramenta, suportada por Java, que traz flexibilidade para o desenvolvedor. Para apresentá-los, dividiremos os assuntos em três partes: polimorfismo em Java; a entidade interface de Java; e interface avançada em Java. Para auxiliar em seus estudos, todo o conteúdo é extensamente baseado em exemplos práticos e contém exercícios para fixação.

Propósito

Preparação

É recomendável que você tenha à sua disposição um computador com kit de desenvolvimento Java (JDK) e uma IDE. Além disso, o acesso à internet também será de grande ajuda para consultar a documentação oficial da linguagem ou tirar dúvidas.

O SDK Java está disponível em na página da Oracle (oracle.com/java/technologies/downloads/). Após baixar e instalar, é recomendável também instalar uma IDE pois, embora não seja imprescindível, isso facilita bastante o desenvolvimento. Além disso, é uma ferramenta muito útil no processo de aprendizagem. Entre as

diversas IDE existentes, o Netbeans (netbeans.apache.org/) e o Eclipse (eclipse.org/downloads/) estão entre as mais utilizadas.

Objetivos

Módulo 1

Polimorfismo em Java

Descrever polimorfismo em Java.

Módulo 2

A entidade de interface em Java

Descrever a criação e uso de interfaces em Java.

Módulo 3

Interface avançada em Java

Reconhecer conceitos avançados de interfaces em Java.



Introdução

Provavelmente, muitos dos que estão lendo estas linhas, se não todos, não têm idade para lembrar dos motivos que levaram ao desenvolvimento da programação orientada a objetos (OO). É quase certo que o evento tecnológico relevante mais próximo

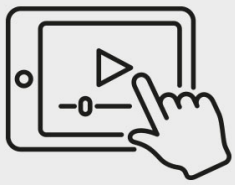
seja o bug do milênio, quando o mundo esteve preocupado com a forma como os sistemas lidariam com o ano 2000 (conhecido como Y2K). No entanto, antes desse marco, o mundo da Tecnologia da Informação (TI) já havia enfrentado outro desafio, este sim grave e de significativo impacto: a crise do software.

Quando surgiram os computadores eletrônicos digitais, os recursos computacionais eram escassos. A complexidade do software não vinha apenas do seu tamanho, mas da necessidade de se utilizar linguagens de baixo nível, como o assembly. Com o desenvolvimento tecnológico, os softwares foram ficando maiores e mais complexos. No final da década de 1960 e ao longo da década de 1970, quando o paradigma de programação estruturada era popular, ocorreu a crise do software.

O que ocorreu naquele período foi que os sistemas se tornaram muito complexos e a forma de codificar, apesar da existência de metodologias, como a análise estruturada, não conseguiu lidar satisfatoriamente com a manutenibilidade e evolução dos softwares. A manutenção de um sistema, muitas vezes, era mais cara do que a construção de um novo.

A resposta à crise foi a programação orientada a objetos (OO), centrada no reuso de código e na manutenibilidade do software. Aqui, o polimorfismo e as interfaces em Java ganham destaque. Nas próximas linhas, vamos explorar esses conceitos, entender como aplicá-los e desvendar seu mecanismo de funcionamento. Como desenvolvedor, conhecer esses conceitos não é apenas uma obrigação, você deve conhecê-los bem e dominá-los.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Material para download

Clique no botão abaixo para fazer o download do conteúdo completo em formato PDF.

Download material



1 - Polimorfismo em Java

Ao final deste módulo, você será capaz de descrever polimorfismo em Java.

Classes e métodos abstratos

Neste tópico, vamos aprofundar o estudo do polimorfismo, introduzindo os conceitos de classes e métodos abstratos, e entenderemos como a abstração de classes funciona em Java. Estamos particularmente interessados em descobrir como esse conceito impacta o polimorfismo e como é aplicado em uma hierarquia de herança. Por sinal, o polimorfismo é uma ferramenta fundamental, que todo programador orientado a objetos precisa dominar. Prepare sua IDE e vamos aos códigos!

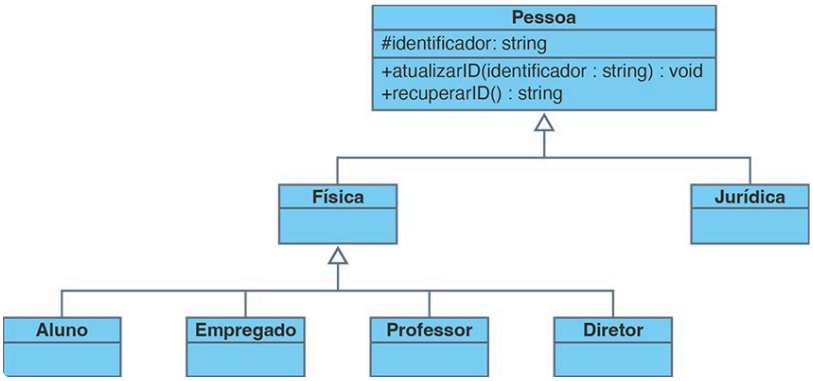
Assista ao vídeo e conheça o conceito de classe e métodos abstratos, veja as implicações deste conceito na instanciação de objetos.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Classes e métodos abstratos

No nosso estudo de hierarquia de herança e seus desdobramentos, constantemente voltamos aos conceitos de especialização e generalização. Quanto mais subimos na hierarquia, tanto mais esperamos encontrar modelos genéricos. A generalização de modelos corresponde à generalização dos comportamentos implementados pelas superclasses. Vimos isso ocorrer no modelo mostrado na imagem a seguir, no qual a classe “Pessoa” representa a generalização de mais alto nível dessa hierarquia.



Especialização de comportamento.

A classe “Pessoa” modela um comportamento genérico. Veja, como exemplo, o caso do método “atualizarID”, mostrado no Código 1. Para a classe “Pessoa” nenhuma crítica é feita ao se atualizar o atributo “identificador”, deixamos a modelagem e a validação nas classes derivadas, que são “Física” e “Jurídica”. Essas classes deverão implementar as validações necessárias, que só fazem sentido para as pessoas física ou jurídica.

JAVA

Código 1: Código parcial da classe "Pessoa" - métodos genéricos.

Apesar de "atualizarID" ser uma generalização de comportamento, se você refletir bem, notará que a implementação é provavelmente inútil. Qualquer caso de instanciação num caso real deverá fazer uso das versões especializadas. Mesmo no caso de um aluno que ainda não possua CPF, é mais razoável prever isso no comportamento modelado pela versão especializada de "atualizarID" da subclasse "Pessoa", pois esse aluno é uma pessoa física. Podemos esperar que, em algum momento, ele venha a ter um CPF, e se usar essa abordagem, manteremos a coerência, evitando instanciar objetos da classe "Pessoa".

Aliás, pensando em termos de flexibilidade, o que esperamos de “Pessoa” é que ela defina o contrato comum à todas as classes derivadas e forneça-nos o comportamento comum que não necessite de especialização. Por exemplo, o comportamento de “atualizarNome” (vide Código 2) é um comportamento que não esperamos precise de ser especializado, afinal pessoas físicas e jurídicas podem ser nominadas da mesma forma. Se conseguirmos isso, teremos uma classe extremamente útil e poderemos tirar o máximo proveito da OO.

JAVA

Código 2: Declaração de classe.

Possivelmente a primeira sugestão que você pensou é simplesmente fornecer o método "atualizarID" vazio em "Pessoa". Há, contudo, um problema com essa abordagem. Imagine que desejamos que o método "atualizarID" retorne "true" quando a atualização for bem-sucedida e "false" no caso contrário. Nesse caso, Java obriga que seja implementado o retorno ("return") do método. Além disso, há outra consideração importante. Se concebemos o nosso modelo esperando que a superclasse nunca seja instanciada, seria útil que houvesse uma maneira de impedirmos isso, evitando assim um uso não previsto de nosso código.

> ## A solução para o problema apontado é a classe abstrata.

Uma classe abstrata é aquela que não admite a instanciação de objetos. Em oposição, classes que podem ser instanciadas são chamadas concretas. O propósito de uma classe abstrata é justamente o que mencionamos antes, fornecer uma interface e comportamentos (implementações) comuns para as subclasses. A linguagem Java implementa esse conceito através da instrução "abstract".

Em Java, uma classe é declarada abstrata pela aplicação do modificador "abstract" na declaração. Isto é, podemos declarar a classe "Pessoa" como abstrata simplesmente fazendo public abstract class Pessoa { ...}. Já se a instrução "abstract" for aplicada a um método, tal método passa a ser abstrato. Isso quer dizer que ele não pode possuir implementação, pois faz parte do contrato (estrutura) fornecido para as subclasses. Nesse caso, a classe deverá obrigatoriamente ser declarada abstrata também. No exemplo em questão, transformamos "atualizarID" em abstrato declarando-o protected abstract void atualizarID (String identificador);. Observe que agora o método não pode possuir corpo.

Uma nova versão da classe “Pessoa” é mostrada no código 3. Nessa versão, o método “atualizarID” é declarado abstrato, forçando a declaração da classe como abstrata. Entretanto, o método “recuperarID” é concreto, uma vez que não há necessidade para especializar o comportamento que ele implementa.

JAVA

Código 3: classe "Pessoa" declarada como abstrata.

Em Java, o efeito de declarar uma classe como abstrata é impedir sua instanciação. Quando um método é declarado abstrato, a sua implementação é postergada. Esse método permanecerá sendo herdado como abstrato até que alguma subclasse realize sua implementação. Isso quer dizer que a abstração de um método se propaga pela hierarquia de classes. Por extensão, uma subclasse de uma classe abstrata será também abstrata a menos que implemente o método abstrato da superclasse.

Apesar de mencionarmos que uma classe abstrata fornece, também, o comportamento comum, isso não é uma obrigação. Nada impede que uma classe abstrata apresente apenas a interface ou apenas a implementação. Aliás, uma classe abstrata pode ter dados de instância e construtores. Vejamos o uso da classe abstrata “Pessoa” no seguinte trecho de código:

JAVA

Código 4: polimorfismo com classe abstrata.

A linha 7 do código 4 cria um vetor de referências para objetos do tipo “Pessoa”. Nas linhas 14 e 16 são instanciados objetos do tipo “Fisica” e “Juridica” respectivamente. Esses objetos são referenciados por meio do vetor “ref”. Nas linhas 17 e 18 é invocado o método “atualizarID”, que é abstrato na superclasse, mas concreto nas subclasses. Dessa forma, o sistema determinará em tempo de execução o objeto, procedendo à vinculação dinâmica do método adequado. Esse é um comportamento tipicamente polimórfico. Na linha 17, o método invocado pertence à classe “Física” e na linha 18, à classe “Juridica”.

Atividade 1

Uma classe abstrata em Java é declarada pelo uso do modificador abstract. Analise as opções a seguir e marque a correta sobre o assunto.

A Se declararmos uma classe como abstrata, seus métodos também deverão ser declarados abstratos.

B Uma classe abstrata pode estender uma classe concreta.

C Uma classe abstrata admite herança múltipla se as superclasses também forem abstratas.

D Uma classe concreta não pode possuir referência para uma classe abstrata.

E Um método abstrato não pode ser herdado.

Parabéns! A alternativa B está correta.

Uma classe abstrata não permite a instanciação, mas não altera as regras de herança. Assim, ela pode tanto estender uma classe concreta quanto ser estendida por uma.

Métodos e classes “final”

Como estamos falando sobre como os modificadores afetam a herança, convém entendermos de que forma as variáveis da superclasse e da subclasse se relacionam. Esse entendimento é crucial para que o programador evite erros conceituais que podem ser difíceis de identificar, e que levam o software a se comportar de maneira imprevisível. A seguir, vamos entender o que são métodos e classe “final”.

Assista ao vídeo e conheça o conceito de polimorfismo, bastante ligado ao de herança, bem como os métodos especializados que implementam comportamentos específicos nas subclasses! Intrigado com o limite disto tudo? Descubra a palavra reservada final que resolve o problema!

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Compreendendo métodos e classe “final”

Agora, vamos analisar a hipótese: considere o método "recuperarID" mostrado no Código 1. Esse método retorna o identificador na forma de uma "String". Esse comportamento serve tanto para o caso de "Fisica" quanto de "Juridica". Em verdade, é muito pouco provável que seja necessário especializá-lo, mesmo se futuramente uma nova subclasse for adicionada. Esse é um dos comportamentos comuns que mencionamos antes e, por isso, ele está na superclasse. Mas se não há motivo para que esse método possa ser reescrito por uma subclasse, então é desejável que possamos impedir que isso ocorra inadvertidamente. Felizmente, a linguagem Java fornece um mecanismo para isso. Trata-se do modificador "final".

> O modificador ”final” pode ser aplicado à classe e aos membros da classe.

Diferentemente de "abstract", declarar um método "final" não obriga que a classe seja declarada "final". Porém, se uma classe for declarada "final", todos os seus métodos são implicitamente "final" (isso não se aplica aos seus atributos).

Métodos "final" não podem ser redefinidos nas subclasses. Dessa forma, se tornarmos "recuperarID" "final", impediremos que ele seja modificado, mesmo por futuras inclusões de subclasses. Esse método permanecerá imutável ao longo de toda a hierarquia de classes. Métodos "static" e "private" são implicitamente "final", afinal não poderiam ser redefinidos de qualquer forma.

Comentário

O uso de "final" também permite ao compilador realizar otimizações no código em prol do desempenho. Como os métodos desse tipo nunca podem ser alterados, o compilador pode substituir as invocações pela cópia do código do método, evitando assim desvios na execução do programa. Vimos que, quando aplicado à classe, todos os seus métodos se tornam “final”. Isso quer dizer que nenhum deles poderá ser redefinido. Logo, não faz sentido permitir que essa classe seja estendida e a linguagem Java proíbe que uma classe “final” possua subclasses. Contudo, ela pode possuir uma superclasse.

Quando aplicada a uma variável, “final” irá impedir que essa variável seja modificada e exigirá sua inicialização, que pode ser feita junto da declaração ou no construtor da classe. Entretanto, uma vez inicializada, qualquer tentativa de modificar seu valor irá gerar erro de compilação.

Como podemos ver no Código 5, uma classe abstrata pode ter membros “final”. A variável dias_letivos foi declarada “final” (linha 9) e por isso precisa ser inicializada, o que é feito na linha 13, no construtor da classe. A não inicialização dessa variável, assim como a tentativa de alterar seu valor irão gerar erro em tempo de compilação.

JAVA

Código 5: Uso de "final".

O método calcularFrequencia (linha 17) foi declarado “final”, não podendo ser redefinido. Entretanto, a classe não é “final” e pode, então, ter classes derivadas.

Atividade 2

Considere o trecho de código a seguir:

JAVA

O método “atualizarAtributoNome” atualiza o atributo “nome” de tipo String da classe Escola. Nesse sentido, é correto afirmar que

A a linha 7 irá gerar erro de compilação, pois “ref” é final.

B a linha 8 irá criar uma instância que não é final.

C a linha 8 irá gerar um erro de compilação, pois “ref” é final.

D a linha 3 irá gerar erro de compilação, pois está instanciando uma variável final.

E as linhas 7 e 8 precisam ser invertidas para o programa compilar.

Parabéns! A alternativa C está correta.

A variável “ref” é uma referência para um objeto do tipo Escola. Ela é final, não o objeto. Assim, ela precisa ser inicializada e não pode referenciar outra instância. Portanto, a tentativa realizada na linha 8 gera um erro de compilação.

Atribuições permitidas entre variáveis da superclasse e subclasse

Não é possível compreender a hierarquia de classes sem compreender o mecanismo que regula a herança e a acessibilidade entre os membros da superclasse e da subclasse. Agora, vamos estudar exatamente como as restrições de visibilidade afetam a interação. Apresentaremos os conceitos de downcasting e upcasting e usaremos um exemplo prático para sedimentar esses novos conhecimentos.

Assista ao vídeo e saiba como a relação subclasse/superclasse interfere na atribuição de valores e no acesso a métodos da hierarquia de classes.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



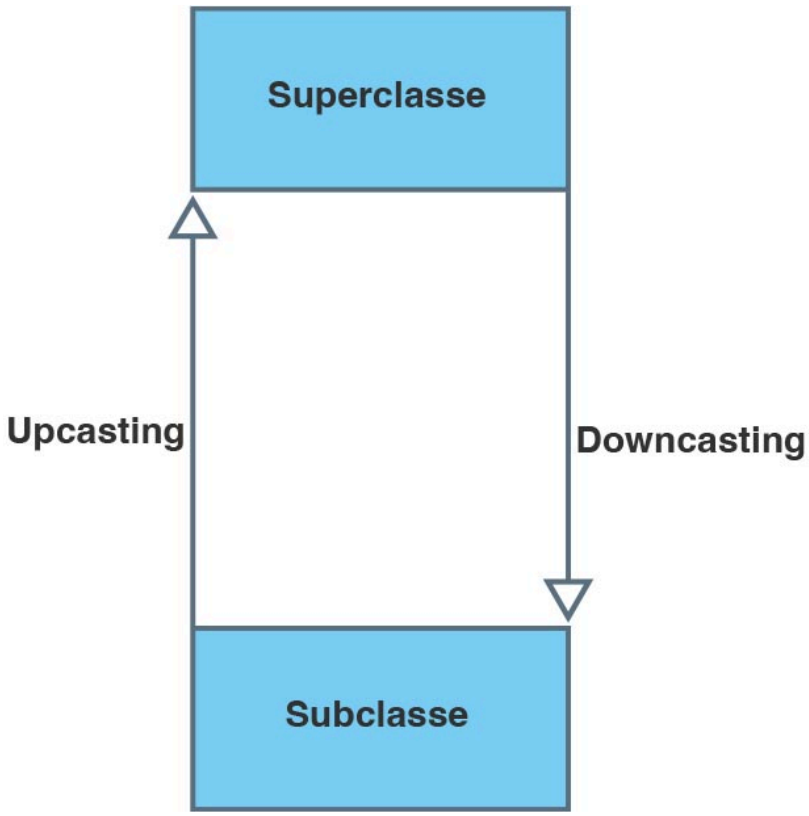
Atribuições permitidas entre variáveis de superclasse e subclasse

Inicialmente, devemos lembrar que, quando uma variável é declarada "private", ela se torna diretamente inacessível para as classes derivadas. Nesse caso, elas são implicitamente "final" e podem, ainda, ter seu valor alterado. Entretanto, isso somente pode ocorrer por meio de métodos públicos ou privados providos pela superclasse. Isso pode parecer uma contradição, mas na verdade não é. As atualizações, feitas pelos métodos da superclasse, ocorrem no contexto dela, no qual a variável não foi declarada "final" e nem é implicitamente tomada dessa forma.

> **Podemos acessar membros públicos ou protegidos da classe derivada a partir da superclasse ou de outra parte do programa fazendo downcasting da referência.**

O downcasting leva o compilador a reinterpretar a referência empregada como sendo do tipo da subclasse. Obviamente, isso é feito quando a variável de referência é do tipo da superclasse. Simetricamente, o upcasting causa a reinterpretação de uma variável de referência da subclasse como sendo do tipo da superclasse. A imagem a seguir ilustra o que acabamos de dizer.

O caso de variáveis privadas é óbvio. Vejamos o que ocorre com as variáveis públicas e protegidas. Como no escopo da herança, ambas se comportam da mesma maneira, ou seja, são plenamente acessíveis entre as classes pai e filha, trataremos apenas de variáveis protegidas. O mesmo raciocínio se aplicará às variáveis públicas.



Reinterpretação de referência.

Para esclarecer o comportamento das variáveis, usaremos as classes mostradas no Código 6 e Código 7.

JAVA

Código 6: Classe Base.

JAVA

Código 7: Classe Derivada.

Já o código 8 implementa o programa principal.

JAVA

Código 8: Variáveis protegidas na hierarquia de classes.

Esse código tem como saída:

JAVA

Código 8: Variáveis protegidas na hierarquia de classes.

Primeiramente, podemos ver que o código mostra que “var_base” representa uma variável compartilhada por ambas as classes da hierarquia. Então, a mudança por meio de uma referência para a subclasse afeta o valor dessa variável na superclasse, o que é esperado pois trata-se do mesmo espaço de memória. A linha 6 do código 8 deixa isso claro. Ao instanciar o objeto do tipo “Derivada”, o seu construtor é chamado. A primeira ação é imprimir o valor da variável “var_base” da superclasse. Como a classe pai é instanciada primeiro, “var_base” assume valor -1. Em seguida, é sobrescrita na classe derivada com o valor -2 e uma nova impressão do seu conteúdo confirma isso.

A linha 9 do código 8 chama o método “atualizarVarSub”, que atualiza o valor de “var_der” da subclasse fazendo o downcasting do atributo (linha 13 do código 6). As impressões de “var_der” confirmam que seu valor foi modificado.

Atividade 3

Julgue as afirmações a seguir:

I – Ao realizar o dowcasting, muda-se o tipo do objeto instanciado, que passa a incorporar os métodos e atributos da classe destino.
II – Pode-se realizar dowcasting e upcasting para tipos definidos fora da hierarquia de classe.
III – O upcasting de uma referência para objeto de uma classe derivada não permite invocar métodos private da superclasse por meio dessa referência.

Analisando as afirmativas, está correto apenas o que se afirma em

A I.

B II.

C III.

D I e II.

E II e III.

Parabéns! A alternativa C está correta.

As operações de downcasting e upcasting não mudam o objeto, apenas modificam a referência, fazendo com que aquele objeto seja interpretado pelo compilador como se fosse de um subtipo ou supertipo, respectivamente. E a definição dessas operações é que isso sempre ocorra dentro da hierarquia. É possível realizar a reinterpretação para objetos fora da hierarquia, mas nesse caso trata-se apenas de casting. Por fim, nenhuma dessas operações altera as restrições de acesso, portanto, o que é privado continua inacessível fora do contexto em que foi declarado.

Usando o polimorfismo

Vamos agora demonstrar o polimorfismo em Java por meio de um programa simples. Para isso, faremos uso de 4 classes: Principal, responsável pelas operações de criação de objetos e demais passos interativos; Pessoa, uma classe abstrata que forma a base de nossa hierarquia; Fisica e Juridica, especializações que concretizam Pessoa.

O polimorfismo é um importante atributo do paradigma OO, e ocorre quando um método é invocado por meio de uma referência para a superclasse, e a versão especializada desse método é executada, com a

resolução ocorrendo em tempo de execução. O uso de polimorfismo contribui para a manutenibilidade do código, pois modificações futuras têm impacto restrito.

Agora, imagine que você é um desenvolvedor trabalhando em um software CAD para arquitetura. Sua tarefa envolve lidar com a construção de figuras planas. Inicialmente, você definiu uma classe Forma, que contém os elementos gerais que todos os objetos derivados precisam ter (cor da borda, cor do preenchimento, método para atualizar a cor da borda, método para atualizar a cor de preenchimento etc.). Você também pode modelar nessa classe os comportamentos específicos, como o cálculo da área, o que pode ser feito por meio de uma instrução switch-case. Entretanto, nesse caso, sempre que você definir uma nova forma, será preciso alterar o código da superclasse.

O uso do switch-case pode atingir o mesmo objetivo, mas a complexidade do código tende a crescer. Essa situação é elegantemente resolvida pelo uso do polimorfismo. Aplicando essa técnica, cada objeto detém o conhecimento sobre aquilo que lhe é específico (o que é preconizado pela OO). Assim, se você definir outra forma, basta codificar nessa classe a maneira de calcular a área e deixar que o programa decida, em tempo de execução, qual método invocar. Ou seja, você passa a se concentrar no que é geral, deixando que a execução resolva como especializar as chamadas.

Assista ao vídeo e acompanhe um exemplo prático do emprego do polimorfismo, o exemplo apresentado vai te ajudar a executar a tarefa proposta a seguir.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Roteiro de prática

Empregando polimorfismo

Na listagem a seguir, vemos o código da classe Pessoa (Código 9). Para nosso exemplo, vamos nos concentrar apenas nos métodos que são relevantes.

JAVA

Código 9: Classe Pessoa.

Perceba que Pessoa é uma classe abstrata, ou seja, não admite instanciação. Na linha 14, definimos o método “recuperarNome”, e nas linhas 18 e 19 estão declarados dois métodos abstratos que, por este motivo, não admitem implementação, cabendo sua implementação à classe que concretizar Pessoa. No nosso caso, as classes Fisica (Código 10) e Juridica (Código 11) concretizam pessoa, como veremos a seguir.

JAVA

Código 10: Classe Fisica.

JAVA

Código 11: Classe Juridica.

Em ambas as classes, você notará que foram implementados os métodos abstratos. Em cada caso, para distinguirmos a execução, o método “recuperarNome” antepõe um prefixo: <NOME>, para objetos da classe Fisica e <RAZAO SOCIAL>, para objetos da classe Juridica.

O método “recuperarID”, além de retornar o identificador, o formata conforme o tipo de objeto. Assim, objetos da classe Fisica apresentam o identificador, com 11 dígitos, formatado com 3 grupos de 3 dígitos separados por ponto, e dois dígitos finais separados por um traço. Já para objetos da classe Juridica, o método formata os 14 dígitos seguindo o padrão de CNPJ: 2 grupos de 3 dígitos e 1 grupo de 2 dígitos, todos separados por ponto. Em seguida, separação por uma barra (/), seguida de 4 dígitos, um traço e outro grupo de dois dígitos.

Portanto, nossa implementação segue exatamente o que havíamos dito. Cada classe que especializa Pessoa contém a codificação dos comportamentos que lhes são peculiares. Vejamos a saída para a execução da classe Principal (Código 12) a seguir.

JAVA

Código 12: Classe Principal.

A saída para a execução do programa é:

JAVA

Saída da execução do programa.

Se observarmos as linhas 3 e 6 da classe Principal, notaremos que criamos um vetor de 6 posições para referências a objetos da classe Pessoa. No entanto, como evidenciado nas linhas 8, 10, 12, 14, 16 e 18, cada elemento do vetor é um objeto das classes derivadas (Fisica ou Juridica). Ao executarmos a linha 20, o vetor é percorrido, e em cada iteração, são invocados os métodos “recuperarNome” e “recuperarID”. O sistema, então, resolve em tempo de execução qual versão correta desses métodos deve ser vinculada, permitindo que cada objeto imprima a informação seguindo sua implementação.

Deixamos como sugestão um pequeno experimento. Faça um clone das classes e transforme a classe Pessoa em uma classe concreta. Forneça para o método “recuperarID” uma implementação em Pessoa que permita distinguir sua execução das versões especializadas. Na classe Principal, repita as mesmas ações para as classes clonadas, execute o

programa e avalie o resultado. Busque entender o que aconteceu. Se necessário, volte aos núcleos teóricos.

Saiba mais

Outras técnicas permitem obter o efeito do polimorfismo, como o uso de estruturas “switch-case”. Entretanto, essa não é a forma OO de se resolver tais situações. Essas formas alternativas trazem outros impactos que devemos conhecer. Assim, faça uma busca na Internet sobre polimorfismo e formas alternativas e veja os prós e contras dessas outras formas. De maneira geral, você perceberá que as alternativas têm muito apelo quando estamos lidando com linguagens estruturadas, como a C, mas desejamos a flexibilidade de conceitos da OO. Mas, para uma linguagem OO, há razão para se usar outras soluções que não o polimorfismo?

Faça você mesmo!

Um estudante, desejando aprofundar seu conhecimento sobre polimorfismo, escreveu os códigos a seguir:

JAVA

JAVA

JAVA

JAVA

Ele deseja que a saída seja a mostrada a seguir:

JAVA

Para que o programa exiba o comportamento polimórfico que o estudante espera, os espaços da linha 3 do código da classe Principal devem ser, respectivamente

A Base e Base.

B Base e Derivada2.

C Derivada1 e Derivada2.

D Derivada1 e Derivada1.

E Derivada2 e Derivada2.

Parabéns! A alternativa A está correta.

O comportamento polimórfico almejado é possibilitado por meio de referências do tipo da classe Base. Ou seja, para que ocorra o resultado esperado, o vetor deve ser de referências de objetos da classe Base.



2 - A entidade Interface de Java

Ao final deste módulo, você será capaz de descrever a criação e uso de interfaces em Java.

Entendendo a entidade "Interface"

Estudaremos agora um artefato da linguagem Java chamado de Interfaces. Esse é outro conceito de OO suportado pela linguagem. Para conduzirmos nosso estudo vamos, primeiramente, entender o conceito e a entidade “Interface” da linguagem Java e, em seguida, estudaremos com maiores detalhes os seus aspectos e vamos explorar seu uso.

Assista ao vídeo e conheça o conceito de interface, que é intuitivo para quem é da área de computação, porém este conceito tem aplicações específicas em OO e, consequentemente, na linguagem Java.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Conceito e exemplo da entidade “Interface”

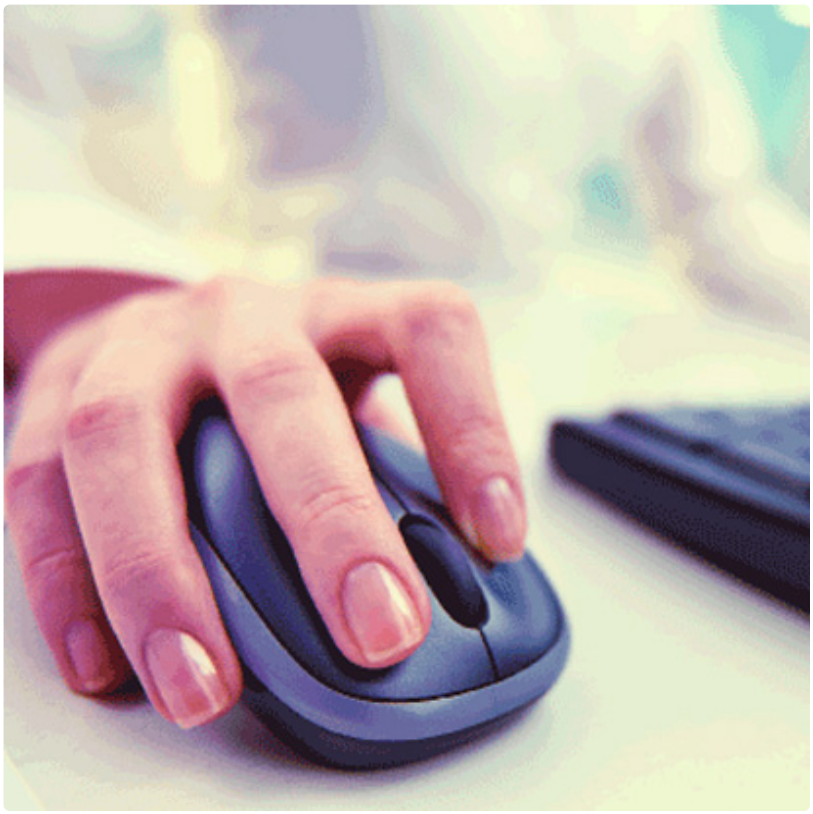
Primeiramente, vamos entender o conceito de interface. Uma interface é um elemento que possibilita a conexão entre dois sistemas de natureza distintas que não se conectam diretamente. Um teclado fornece uma interface para conexão homem-máquina, sendo um bom exemplo. Outro exemplo clássico são as telas gráficas de um programa, chamadas *graphic user interface* (GUI) ou interface gráfica de usuário, que permitem a ligação entre o usuário e o software.

Como sabemos, o paradigma de programação orientada a objetos busca modelar o mundo real por meio de entidades virtuais, e as classes e objetos são entidades que fazem isso. A interface tem o mesmo propósito.

Podemos exemplificar da seguinte forma: considere um mouse ótico com fio, três botões e uma roda. Esse instrumento é usado para controlar o computador, oferecendo os mecanismos que mencionamos. Podemos movê-lo no plano, e o sensor de movimento transformará esse movimento em deslocamentos do ponteiro na tela. Podemos pressionar cada um dos três botões individualmente ou de maneira combinada, além de girar a roda para frente e para trás. Todas essas ações físicas são transformadas pelo mouse em comandos e enviadas ao computador. Essa é toda a possibilidade de interação física com o mouse. Aqui, a interface de interação com o mouse é exatamente o que acabamos de descrever. Ela é uma especificação das interações possíveis com o dispositivo, mas não define o comportamento. Por exemplo, pressionar o botão direito para interagir desencadeará um comportamento específico, dependendo do programa em uso.

Trazendo nosso exemplo para o software, podemos criar um mouse virtual para fazer simulações, por exemplo. Nesse caso, para que nosso mouse represente bem o modelo físico, devemos impor que sua interface de interação com o usuário seja a mesma. Dessa forma, independentemente do tipo de mouse modelado (ótico, laser, mecânico), todos terão que oferecer a mesma interface: detecção de movimento, detecção de pressionamento dos botões 1, 2 e 3 e detecção de

movimento da roda. Cada mouse simulará sua própria versão do mecanismo que realiza tais ações, mas todos oferecerão a mesma interface ao usuário.



Modelo físico de mouse em uso.

Podemos dizer, então, que uma interface no paradigma OO é uma estrutura que permite garantir certas propriedades de um objeto. Ela possibilita definir um contrato padrão para a interação que todos devem seguir, isolando os detalhes de implementação do mundo exterior. Para o nosso exemplo, uma definição de interface pode ser dada pelo conjunto de métodos a seguir. Qualquer programador que deseje estender nosso mouse deve obedecer a esse contrato.

JAVA

Conjunto de métodos para definição da interface.

Atividade 1

Interfaces são um conceito usado em programação orientada a objetos que Java suporta por meio da entidade “Interface”. Sobre essa entidade de Java é correto afirmar que

A Admitem herança múltipla.

B Métodos e atributos protegidos são legados às subinterfaces.

C Só podem possuir atributos privados.

D Estendem apenas classes concretas e interfaces.

E Não permitem atributos públicos.

Parabéns! A alternativa A está correta.

A interface é uma das formas que Java contorna a restrição à herança múltipla. Enquanto as classes em Java não admitem essa situação, as interfaces aceitam herança múltipla.

Particularidades da entidade “Interface”

Em C++, o conceito de interfaces é suportado por meio de classes virtuais puras, também chamadas classes abstratas. A linguagem Java possui uma entidade específica chamada “Interface”. Esse é um recurso muito utilizado e de conhecimento obrigatório por todo programador que deseja desenvolver seguindo o paradigma OO. Por isso, vamos agora nos debruçar sobre esse assunto.

Assista ao vídeo e verifique que o conceito de interface não é exclusivo de Java, porém tem particularidades na linguagem Java. Explore estas particularidades!

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Características da “Interface”

A sintaxe para se declarar uma interface em Java é muito parecida com a declaração de classe: “public interface Nome { ...}”.

No entanto, uma interface não admite atributos e não pode ser instanciada diretamente. Ela é um tipo referência e somente pode conter constantes, assinaturas de métodos, tipos aninhados, métodos estáticos e default. Apenas os métodos default e estático podem possuir implementação. Tipicamente, uma interface declara um ou mais métodos abstratos, que são implementados pelas classes.

Comentário

Interfaces, assim como as classes, permitem herança, mas a hierarquia estabelecida se restringe às interfaces. Uma interface não pode estender uma classe e vice-versa. Contudo, as interfaces, diferentemente das classes admitem herança múltipla, o que significa que uma interface pode derivar de duas superinterfaces (interfaces pai). Nesse caso, a interface derivada herdará todos os métodos, constantes e outros tipos membros da superinterface, exceto os membros que ela sobrescreva ou oculte.

Uma classe que implemente uma interface deve declarar isso explicitamente pelo uso do modificador “implements”. É possível a uma classe implementar mais de uma interface. Nesse caso, as interfaces implementadas devem ser separadas por vírgula. Um exemplo de implementação de interfaces por uma classe é mostrado a seguir:

JAVA

Código 13: Interface "Identificador".

JAVA

Código 14: Interface "iPessoa".

Veja agora a implementação das interfaces "Identificador" e "iPessoa" pela classe "Pessoa".

JAVA

Código 15: Implementação das interfaces "Identificador" e "iPessoa" pela classe "Pessoa".

Observe que na linha 1 do código 15 é declarado que a classe “Pessoa” implementará as interfaces “iPessoa” e “Identificador”. Quando uma classe implementa uma ou mais interfaces, ela deve implementar todos os métodos abstratos das interfaces. E de fato, é o que ocorre. A classe, contudo, não pode alterar a visibilidade dos métodos da interface. Assim, métodos públicos não podem ser tornados protegidos por ocasião da implementação.

Reflexão

Lembre-se de que uma Interface define um contrato. Sua finalidade é proporcionar a interação com o mundo exterior. Por isso, não faz sentido restringir o acesso aos métodos da interface. Ou seja, em uma interface, todos os métodos são públicos, mesmo se omitirmos o modificador “public”, como no código 13 e código 14. Além disso, em uma interface, todas as declarações de membro são implicitamente estáticas e públicas.

Em uma Interface a declaração de um membro oculta toda e qualquer declaração desse membro nas superinterfaces. Por exemplo, se uma interface derivada de “Identificador” declarar um método com o nome “validarID”, a correspondente declaração desse método no Código 13

tornar-se-á oculta. Assim, uma interface herda de sua superinterface imediata todos os membros não privados que não sejam escondidos.

Podemos fazer declarações aninhadas, ou seja, declarar uma interface no corpo de outra. Nesse caso, temos uma interface aninhada. Quando uma interface não é declarada no corpo de outra, ela é uma interface do nível mais alto (top-level interface). Uma interface também pode conter uma classe declarada no corpo.

Observando o código 13, vemos na linha 2 que a interface possui um atributo estático (“tamanho_max”). Como vimos, esse é um caso de atributo permitido em uma interface. Observe também que não é possível declarar os métodos com o corpo vazio em uma interface. Imediatamente após a assinatura, a cláusula deve ser fechada com ; (ponto e vírgula). A Java também exige que sejam utilizados identificadores nos parâmetros dos métodos, não sendo suficiente informar apenas o tipo.

> Além do tipo normal de interface, existe o Annotation, que é um tipo especial de interface.

A Interface tipo Annotation, permite a criação de tipos de anotações pelo programador. Esse tipo de interface é declarado precedendo-se o identificador “interface” com o símbolo “@”, como nesse exemplo: “@interface Preambulo {...}”. Uma vez definido, esse novo tipo de anotação se torna disponível para uso juntamente com os tipos *bult-in*.

Resumindo

Fica claro, por tudo que estudamos até aqui, que as interfaces oferecem um mecanismo para ocultar as implementações. Portanto, elas são um mecanismo que nos permite construir as APIs (*application programming interfaces*) de softwares e bibliotecas. As APIs possibilitam que o código desenvolvido seja disponibilizado para uso por terceiros, mantendo oculta a implementação. Isso é muito útil quando não se deseja compartilhar o código que realmente implementa as funcionalidades, seja por motivos comerciais, de segurança, de proteção de conhecimento ou outros.

A pergunta que você deve estar se fazendo nesse momento é “qual a diferença entre classes abstratas e interfaces?”. Essa é uma ótima pergunta. Uma classe abstrata define um tipo abstrato de dado. Define-se um padrão de comportamento que todas as classes que se valham dos métodos da classe abstrata herdarão da classe que os implementar. Além disso, uma classe abstrata pode possuir estado (atributos) e

membros privados, protegidos e públicos. Isso é consistente com o que acabamos de dizer e muito mais do que uma interface pode possuir. Por si só, essa diferença significa que as classes abstratas podem representar ou realizar coisas que as interfaces não podem. É evidente que uma classe puramente abstrata e sem atributos terminará mimetizando o comportamento de uma interface, entretanto, ainda assim não serão equivalentes, já que uma interface admite herança múltipla e classes não.

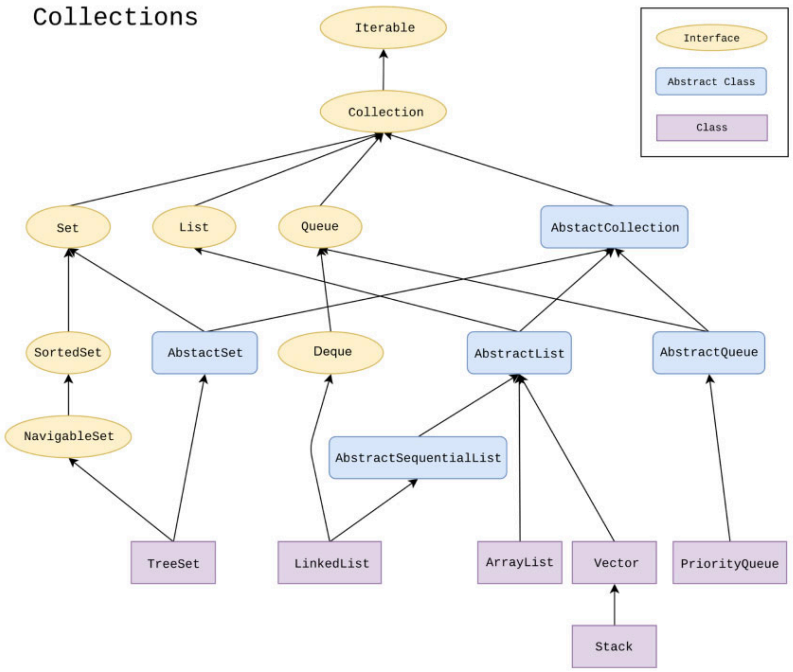
> **Uma interface é uma maneira de se definir um contrato. Se uma classe abstrata define um tipo (especifica o que um objeto é), uma interface especifica uma ou mais capacidades.**

Como visto no caso do código 13 a interface define as capacidades necessárias para se manipular um identificador. Não se trata propriamente de um tipo abstrato, pois tudo que ela oferece é o contrato. Não há estado e nem mesmo comportamentos ocultos. Portanto, uma classe que implementar essa interface proverá o comportamento especificado pela interface.

Mas atenção para não confundir a linha 2 com um estado da instância, pois não é! Além dessa variável ser declarada como final, não há métodos que atuam sobre ela para modificar seu estado.

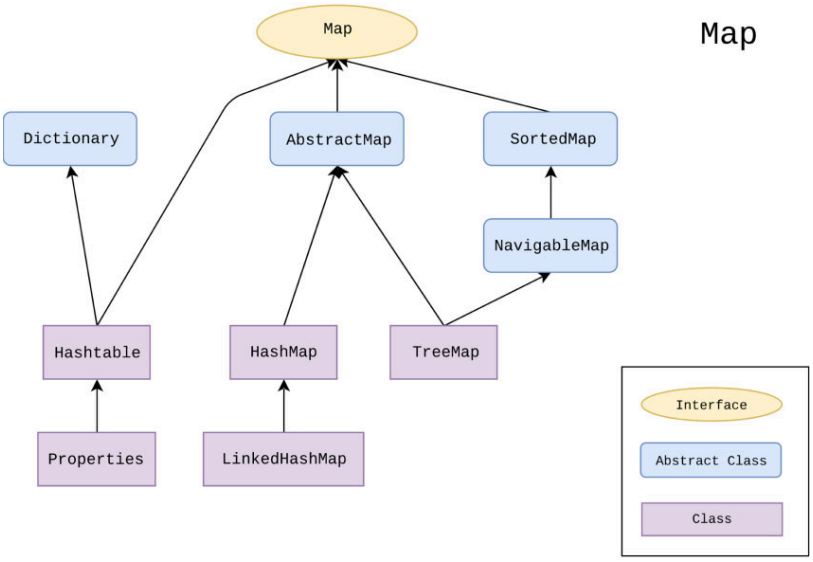
Esclarecidas as diferenças entre as classes abstratas e as interfaces, nos resta saber quando usá-las. A rigor, a discussão é aberta e não existe uma regra. Isso dependerá muito de cada caso, da experiência do programador e da familiaridade com o uso de ambas. Porém, com base nas diferenças apontadas, podemos estabelecer uma regra geral. Quando seu objetivo for especificar as capacidades que devem ser disponibilizadas, a interface é a escolha mais indicada. Quando estiver buscando generalização de comportamento e compartilhamento de código, além de atributos comuns (um tipo abstrato de dado), as classes abstratas surgem como a opção mais adequada.

Veja as imagens a seguir. Ambas mesclam o uso de interfaces e classes abstratas.



Hierarquia de herança – Collections.

Estrutura de hierarquia de herança - Map



Hierarquia de herança – Map.

Na primeira imagens (*Collections*), as interfaces “Set”, “List” e “Queue”, por exemplo, definem as capacidades que devem ser disponibilizadas. Essas capacidades estão ligadas ao contrato que deve ser estabelecido para a estrutura de dados que cada uma se relaciona. As classes abstratas, como “AbstractList”, vão oferecer implementações compartilhadas. No caso de “AbstractList” podemos ver que pelo menos duas classes, “ArrayList” e “Vector” fazem uso dela. Essa imagem, que mostra o modelo das coleções da API Java, é um ótimo exemplo de que os conceitos de classes abstratas e interfaces são complementares e, portanto, não se trata de escolher entre uma e outra.

Atividade 2

Sobre a entidade Interfaces de Java é correto afirmar que:

A Pode estender mais de uma superinterface.

B Os métodos e atributos protegidos são legados às subinterfaces.

C Possui as mesmas limitações de herança que as classes.

D Estende-se apenas às classes concretas e interfaces.

E Interfaces e classes abstratas são mutuamente exclusivos.

Parabéns! A alternativa E está correta.

Interfaces aceitam herança múltipla e, dessa forma, não têm a mesma restrição de herança que classes. Além disso, interfaces podem ser declaradas abstratas.

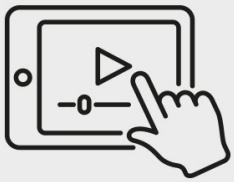
Usando interfaces

Agora que compreendemos o que são interfaces, vamos ver como usá-las. Para isso vamos utilizar a implementação que já realizamos na seção anterior (código 13, código 14 e código 15). Vamos criar uma pequena interação para demonstrar esse uso. Você pode facilmente

aproveitar esses exemplos e modificá-los para explorar o conceito e o uso de interfaces.

Assista ao vídeo e se aprofunde no conceito de interface em Java aplicando o conceito em um exemplo.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Aplicação de interfaces

Vimos a declaração das interfaces “Identificador” e “iPessoa” e da classe “Pessoa”, que as implementa, respectivamente nos códigos 13, 14 e 15. Podemos ver que todos os métodos abstratos de ambas as interfaces foram implementados em “Pessoa”. O método “main” do programa, mostrado no código 16, nos permite explorar seu uso.

JAVA

Código 16: Trabalhando com interfaces.

Nas linhas 3 e 4 do Código 16 declaramos duas variáveis, uma que faz referência à variável “Identificador” e outra à variável “iPessoa”. Já afirmamos que uma interface não admite instanciação direta, entretanto, não fizemos nenhuma restrição quanto a ter uma variável declarada como referência para a interface. De fato, na linha 7, não

estamos instanciando a interface, mas sim a classe “Pessoa”. O objeto instanciado, contudo, será referenciado pela variável “refIdt”.

Curiosidade

Podemos usar uma variável do tipo da interface para referenciar objetos da classe que a implementa. Entretanto, não é suficiente que a classe forneça o comportamento dos métodos abstratos, ela precisa declarar explicitamente que implementa a interface.

Uma vez instanciado o objeto, podemos invocar o comportamento dos métodos especificados por “Identificador”. Entretanto, nesse caso, somente esses métodos e aqueles que a interface “Identificador” herda estarão disponíveis. Nenhum dos métodos especificados em “iPessoa”, apesar de implementados em “Pessoa”, ou mesmo outros métodos implementados nela estarão acessíveis por meio dessa referência (“refIdt”). Por isso, descomentar a linha 10 irá gerar erro de compilação.

Não é possível fazermos a atribuição de “refIdt” para “refiPessoa”, pois os tipos não são compatíveis. Todavia, se fizermos um typecasting, como o mostrado na linha 12, a atribuição se torna possível. Agora, podemos usar a variável “refiPessoa” e acessar os métodos dessa interface que foram implementados em “Pessoa”. Evidente que, caso tivéssemos usado uma variável do tipo “Pessoa” para referenciar o objeto instanciado, todos os métodos estariam prontamente acessíveis.

A saída do código 16 é:

JAVA

Saída do código 16.

Atividade 3

Sobre interfaces em Java, assinale a única opção correta:

A São equivalentes às classes abstratas.

B Interfaces permitem a declaração de funcionalidades e estas devem ser definidas nas classes que a implementam.

C Interfaces permitem definir funcionalidades que podem ser sobrescritadas ou não por quem a implementa.

D Interfaces com múltiplas superclasses não podem ter subinterfaces.

E Interfaces podem estender classes abstratas, mas não as concretas.

Parabéns! A alternativa B está correta.

Uma interface admite apenas a declaração da funcionalidade (método). E todas as funcionalidades declaradas deverão ser definidas (implementadas) na classe que a implementa.

Interfaces em Java na prática

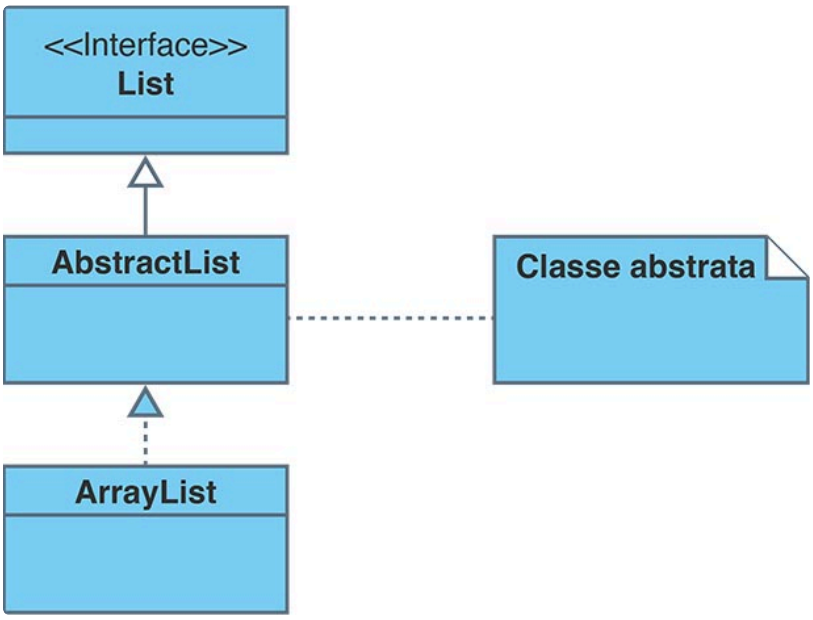
Como vimos, a interface é um conceito muito útil. Por meio dela podemos definir um contrato padrão, ou seja, a interação entre objetos, e isolar os detalhes de implementação. Em outras palavras, com a interface podemos impor a existência de métodos e propriedades em um grupo de classes, sem nos preocuparmos com a implementação desses métodos.

Para nosso estudo prático, vamos buscar um caso real. Por sinal, estudar o código de outros (bons) programadores é uma ótima forma de

aprender. Vamos nos concentrar em um exemplo que seja representativo e capture uma situação real, na qual o uso de interface seja uma solução técnica realmente interessante. Esse exemplo trará uma noção sobre qual contexto tem na interface uma solução adequada.

Para nosso caso prático, nada melhor do que analisarmos o uso de interface na implementação de uma biblioteca da linguagem Java. Estamos falando do framework Collection. Se você consultar a documentação da linguagem Java, verá que a classe “List” é uma interface e a classe “ArrayList” é a classe concreta que implementa os métodos e propriedades.

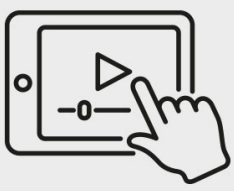
Obviamente, essas classes são extensas, e estudá-las integralmente extrapola nosso objetivo. Por isso, faremos uma análise geral sobre elas, mas nos concentraremos apenas em alguns poucos métodos, explorando os desdobramentos dessa implementação. A seguir, vemos um recorte da hierarquia de Collection, com foco no nosso exemplo.



Visão parcial da hierarquia do framework Collection.

Assista ao vídeo e acompanhe um exemplo de implementação que irá te ajudar a implementar sua própria hierarquia de classes usando o conceito de interface.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Roteiro de prática

Primeiramente, vamos apresentar um exemplo de uso de “ArrayList”. Para isso, vamos nos valer da classe “Principal” (código 17) mostrada na listagem a seguir.

JAVA

Código 17: Classe Principal.

No exemplo, criamos uma estrutura do tipo “ArrayList” para armazenar números inteiros. Nosso programa permite definir o tamanho da lista e realizar algumas operações básicas como inserção, recuperação, atualização e remoção. Essas operações estão definidas, respectivamente, nas linhas 23, 59, 64 e 74. Assim, para inserirmos um elemento na lista, usamos o método “add”, o método “get” para recuperá-lo e o método “set” para atualizá-lo. Já a remoção é feita com “remove”. Vamos tomar esses métodos como exemplo para explorar o uso de interface. Vejamos como eles aparecem na classe “List” (código 18), cuja listagem parcial é mostrada abaixo.

JAVA

Código 18: Listagem parcial da classe List (framework Java Collection).

Podemos identificar a listagem apresentada se trata de uma interface (linha 3). Nela, existe uma lista de operações obrigatórias, que mostramos apenas uma visão parcial. Também vemos presentes os 4 métodos que usamos: “add, remove, set e get”. Todos eles aparecem sem implementação, como era de se esperar. Porém, essa interface definiu o contrato para quem for consumir seus serviços. Sabemos que, ao usar o objeto derivado de uma implementação de “List”, teremos à nossa disposição os métodos que mencionamos, cuja assinatura está declarada em “List”.

Vamos analisar a classe que implementa “List”.

JAVA



Código 19: Listagem parcial da classe ArrayList (framework Java Collection).

Como vemos na linha 7, a classe “ArrayList” (código 19) implementa o comportamento do contrato definido pela interface “List”. Para os métodos que estamos considerando, o comportamento está modelado entre as linhas 48 e 95 da listagem parcial. Caso um programador use esse framework, o contrato foi previamente definido pela interface. E se o programador desejar criar alguma outra estrutura que seja aderente a

esse padrão, ou seja, implemente “List”, ele deverá prover a implementação dos métodos obrigatórios.

Perceba que, dessa forma, a troca de estrutura não provocará alteração no código que a usa. Por exemplo, suponha que um programador crie uma estrutura que mantenha a lista sempre ordenada na inserção. Para isso, ele define a classe “InsertionSortedList”, que implementa “List”. Assim, podemos escolher utilizar a nova estrutura sem precisar alterar nenhuma linha de nosso código da classe Principal (código 17), com exceção, claro, da declaração da nova estrutura.

Faça você mesmo!

Um programador escreveu os códigos mostrados a seguir:

JAVA

JAVA

JAVA

No entanto, quando compilou o código, o compilador acusou erro. Assinale a única opção que corrige corretamente o código.

A Antepor o modificador “final” na linha 2 da definição da interface iTeste.

B Remover a linha 2 da definição da interface iTeste e colocá-la antes da linha 2 da declaração da classe Base.

C Retirar o modificador “abstract” da declaração da interface iTeste.

D Substituir a definição do método na linha 3 da definição da interface iTeste por “;”.

E Na classe Base, alterar “implements” por “extends”.

Parabéns! A alternativa D está correta.

iTeste é uma interface e, por isso, não admite a definição dos métodos que possui, mesmo que essa definição seja vazia. Assim, é necessário retirar o bloco { } da linha 3 da definição da interface iTeste e substituí-lo por ; .



3 - Interface avançada em Java

Ao final deste módulo, você será capaz de reconhecer conceitos avançados de interfaces em Java.

Aninhamento de interfaces e herança

Agora que já estamos ambientados com o uso de interfaces, veremos agora outras características dessas entidades. Também veremos agora o aninhamento de interfaces e seu uso com herança. O aninhamento de interfaces tem as mesmas razões que justificam o aninhamento de classes: o agrupamento de interfaces relacionadas, facilitando a manutenção do código. Todavia, no caso da interface, não é apenas a herança que está com pauta, mas a implementação. Podemos concretizar uma interface aninhada sem explicitamente concretizar a interface externa? Podemos fazê-lo a partir de outro pacote? É o que vamos descobrir a seguir.

Assista ao vídeo e acompanhe, vamos nos aprofundar ainda mais na relação entre o conceito de interface e o conceito de herança, explorando mais esta relação e a aplicação destes importantes conceitos de OO.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Como aninhar interfaces e herança

Vamos começar com o caso mais simples, em que temos dois pacotes: “pctAlfa” e “pctBravo”. O pacote “pctBravo” contém apenas interfaces. A interface “iExterna” é a interface externa que engloba “iInterna” (aninhada). No mesmo pacote temos a interface “iBravo”, que estende a interface “iExterna”. Todas as interfaces possuem o atributo “nome”, refletindo o nome da Interface. Entretanto, apenas a interface “iExterna” possui o método getNome declarado.

O pacote “pctAlfa” possui a interface iAlfa, que também tem o atributo “nome” e estende a interface “iInterna” do pacote “pctBravo”. Já a classe “Concreta” realiza (implementa) a interface “iAlfa”. Observe que ela implementa o método “getNome”. Veja os códigos das classes Principal e Concreta e a Interface iAlfa

JAVA

Código 20: Classes Principal e Concreta.

JAVA

Código 21: Interface iAlfa.

Mas se “getNome” é declarado em “iExterna” e “Concreta” implementa “iInterna”, o que está acontecendo? Será que há relação de herança em Interfaces aninhadas? Vamos explorar essa questão mais adiante, na parte prática. Por fim, a classe “Principal” modela o comportamento do programa. Os códigos das interfaces podem ser vistos a seguir.

JAVA

Código 22: Interface iExterna e Interface aninhada iInterna.

JAVA

Código 23: Interface iBravo.

Observando esses códigos, você deve perceber certa semelhança com o caso de herança de classes aninhadas. Formalmente, apenas interfaces e classes internas são chamadas de aninhadas. Mas cabe fazermos algumas observações: interfaces aninhadas são estáticas (static), mesmo se o modificador não estiver declarado. Isso faz sentido, pois uma interface não pode ter estado e não pode ser instanciada. Assim, uma interface aninhada é um membro estático da interface externa.

> **Se uma interface for declarada dentro de uma classe, ela pode ter qualquer modificador de acesso.**

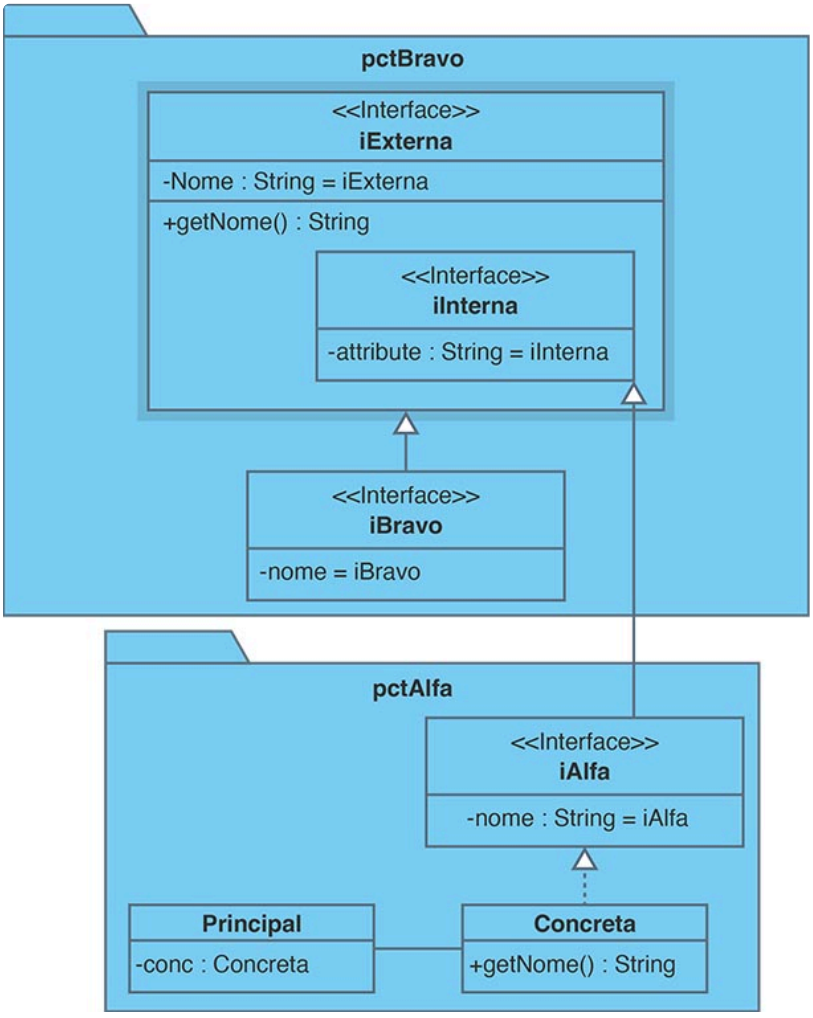
Se uma interface for declarada como membro de outra interface (caso de “iInterna”), entretanto, ela não pode ter qualquer modificador de acesso. Tente aplicar algum modificador de acesso na linha 6 do código 22 diferente de “public” e compile o código. Você verá que o compilador emite uma mensagem de erro. Isso ocorre porque interfaces declaradas como membros de outras interfaces somente podem ser públicas.

Vamos executar o código mostrado nas listagens que vimos e ver o que acontece. Sem a aplicação de modificador de acesso na Interface “iInterna”, o código compila normalmente e a saída será:

JAVA

Resultado da saída do código mostrado sem erro após compilação.

A imagem a seguir representa o caso dos pacotes “pctAlfa” e “pctBravo.



Aninhamento de Interfaces: caso básico.

O nosso exemplo, intencionalmente, mistura dois conceitos: herança e interface. Veja que a classe “Concreta” implementa uma interface de outro pacote (conceito de interface) e “iAlfa” estende a mesma interface (conceito de herança), que também está em outro pacote. Assim, vemos como esses conceitos funcionam para Interfaces em pacotes distintos. Se fizermos a classe “Concreta” implementar “iAlfa”, veremos, sem surpresa alguma, que o programa compila da mesma forma, agora produzindo a seguinte saída.

JAVA

Saída da compilação com implementação da interface “iAlfa” na classe “Concreta”.

De certa forma, a situação é mais simples do que o caso de classes aninhadas, pois interfaces internas têm sempre o modificador de acesso menos restrito de todos: “public”.

Como curiosidade, vale mencionar que o compilador Java cria um arquivo “.class” separado para a interface aninhada. O nome do arquivo será uma combinação do nome da Interface externa e interna, separados pelo símbolo de cifrão (“$”). No nosso caso, o nome do arquivo é “iExterna$iInterna.class”.

Como sabemos, em Java, uma classe pode estender apenas uma outra classe (não existe herança múltipla), entretanto, ela pode implementar mais de uma Interface. Veja!

Na classe “Concreta” (código 20), vamos modificar a linha 3, de maneira que ela também implemente a classe iAlfa (“class Concreta implements iAlfa, iExterna.iInterna {“). Tente compilar o código e veja o que ocorre. Você consegue explicar o que está havendo?

A variável “nome” é definida em “iAlfa” e “iInterna”, de forma que a referência a ela é ambígua. Ainda que você modifique a declaração de “iAlfa” para que ela não mais estenda “iInterna”, o problema persistirá. A desambiguação pode ser feita utilizando a referência completa da variável na linha 5. Por exemplo, “iAlfa.nome” ou “iExterna.iInterna.nome” funcionarão sem gerar erros na compilação.

Atividade 1

Julgue as afirmativas a seguir:

I – Uma classe só pode implementar uma interface aninhada se implementar a classe mais externa.
II – Uma interface que estenda uma interface aninhada, também estende a interface mais externa.
III – Se uma interface estender uma interface aninhada, ela precisará usara referência completa mesmo se ambas estiverem no mesmo pacote.

É correto apenas o que se afirma em:

A I.

B II.

C III.

D I e II.

E II e III.

Parabéns! A alternativa C está correta.

Uma interface aninhada só é acessível por meio da referência à interface externa. Isso ocorre porque a interface aninhada é membro da externa.

Interfaces funcionais

A partir da versão 8 da linguagem Java, foi introduzido o conceito de interface funcional. Não se trata de uma nova entidade, mas de um caso particular de interfaces que suporta a programação funcional. Esse tipo de interface é muito empregado com as capacidades lambda de Java. O termo lambda vem do cálculo lambda, que é uma coleção de sistemas formais usados para simular qualquer máquina de Turing.

A partir de agora, vamos entender o que é programação funcional e compreender como definir e usar interfaces funcionais. Para isso, começaremos apresentando brevemente a capacidade lambda de Java e prosseguiremos nosso estudo nos baseando em uma versão mais simples do programa que utilizamos anteriormente.

Assista ao vídeo e conheça o conceito de interfaces funcionais que nada mais é do que a junção do conceito de interface com o paradigma funcional na linguagem Java.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Java lambda: uma breve explicação

O termo lambda tem origem no cálculo lambda, criado por Alonzo Church em 1930. Conceitualmente, trata-se de um sistema formal capaz de expressar uma máquina de Touring. Esse conceito é naturalmente ligado às linguagens de programação. A Java passou a oferecer capacidades lambda a partir da versão 8 da linguagem, fazendo com que passasse a suportar técnicas de linguagem funcional.

> Uma expressão lambda em Java é uma função implementada anonimamente.

Em Java uma expressão lambda prescinde da nomeação do método que a implementa. Além disso, ela dispensa a declaração de modificadores e tipos de parâmetros e retorno. Não é nosso objetivo aqui detalhar seu uso ou aprofundar seu estudo, mas abordá-la na medida necessária para desenvolvermos os demais conceitos.

Uma expressão lambda tem, em Java, a sintaxe definida por uma lista de parâmetros (que pode ser vazia), o token seta (->) e um corpo. Por exemplo: () -> 17; sempre retorna 17; x -> x/10; divide o valor de x por 10 e retorna o resultado. O código 24 mostra um trecho de código com expressão lambda em Java.

JAVA

Código 24: Expressão lambda em Java.

Programação funcional

A programação funcional é um paradigma de programação baseado na avaliação de funções matemáticas (expressões lambda) que não possuem estados ou dados mutáveis. Trata-se de um paradigma de programação declarativa, fundamentado em árvores de expressões que mapeiam valores em outros valores, diferindo, assim, da programação imperativa tradicionalmente conhecida. Anteriormente, vimos um exemplo de programa declarativo (Código 24).

Comentário

Sem a programação funcional, a única forma de passar alguma funcionalidade por parâmetro em Java seria por meio da criação de uma classe que realize uma interface (quem provê o contrato). Seria, então, necessário instanciar a classe e passar esse objeto como parâmetro. Expressões lambda e interfaces funcionais tornam essa tarefa mais simples e direta.

Novamente, nosso objetivo não é esgotar o assunto, mas contextualizar o conceito de interfaces funcionais. Há muito a ser dito sobre programação funcional, mas os principais livros sobre a linguagem apresentam o assunto em maiores detalhes. A seguir, mostramos um código (código 25) elaborado com o conceito de programação funcional, que converte graus Celsius para Fahrenheit. Repare a expressão lambda na linha 4.

JAVA

Código 25: Programação funcional.

Interfaces funcionais

Em Java, uma interface contendo apenas um método abstrato é chamada interface funcional ou interface SAM (*single abstratct method*). A API java é repleta de interfaces funcionais, como, por exemplo, as Interfaces “Comparator” e “Runnable”. Essa restrição, que consiste em possuir apenas um método abstrato, inclui métodos que sejam herdados da superclasse. Vamos analisar os códigos a seguir.

JAVA

Código 26: Interface iAlfa.

JAVA

Código 27: Interface iBravo.

Nos exemplos que vemos, “iAlfa” (código 26) é uma interface funcional, pois possui um único método abstrato (linha 3). O mesmo vale para “iBravo” (código 27), pois o único método abstrato que ela possui é o que herda de “iAlfa”. No entanto, vemos a seguir, “iCharlie” (código 28) não é uma interface funcional, pois não atende ao critério SAM. Nesse caso, há dois métodos abstratos: “imprimeURL()” e “imprimeNome()”.

JAVA

Código 28: Interface iCharlie.

Interfaces funcionais podem ser anotadas com “@FunctionalInterface”, o que instrui o compilador a verificar se o critério SAM está sendo atendido. Essa anotação também permite informar ao programador o tipo de Interface, deixando claro que ela possui, intencionalmente, apenas um método abstrato. No código mostrado anteriormente, inclua a anotação de interface funcional e observe o resultado.

Atividade 2

Julgue as afirmativas a seguir:

I – Uma Interface Funcional é um tipo de classe abstrata.
II – Em uma hierarquia de herança de Interfaces, na qual a superinterface é do tipo funcional, todas as subinterfaces serão desse tipo, exceto as que possuírem um método abstrato e suas derivadas.
III – A condição para que uma Interface seja do tipo funcional é que ela seja declarada com expressões lambda.

É correto apenas o que se afirma em:

A I.

B II.

C III.

D I e II.

E II e III.

Parabéns! A alternativa B está correta.

A condição para ser uma interface funcional é possuir apenas um método abstrato, incluindo-se os que são herdados. Em uma hierarquia de herança, o método abstrato da superclasse será herdado pelas subclasses, o que as torna, também, interfaces funcionais. Contudo, quando uma subinterface já possuir um método abstrato, ela violará o critério SAM. E como suas derivadas herdam os métodos abstratos dela e da superinterface, elas não são interfaces funcionais.

Explorando os conceitos avançados de interface

Quando falamos sobre interfaces aninhadas, demonstramos a implementação de uma Interface por uma classe de outro pacote. Nessa implementação, utilizamos um método com a mesma assinatura da interface externa e questionamos se isso representava uma relação de herança. Agora, vamos responder a essa pergunta e aproveitar a oportunidade para explorar, na prática, alguns dos conceitos estudados até aqui.

Nosso estudo será baseado em uma versão um pouco modificada dos códigos que usamos ao falar de aninhamento de interfaces. As versões modificadas estão a seguir. Inicialmente, manteremos as classes “Principal” e “Concreta” como estavam. Vamos, também, adicionar mais uma classe “Base” ao pacote “pctBravo”, na qual definiremos uma interface aninhada chamada “iNested”.

JAVA

Código 29: Classes Principal e Concreta.

JAVA

Código 30: Interface iExterna e Interface aninhada iInterna.

Assista ao vídeo e acompanhe um exemplo prático que vai te ajudar a implementar a “interface” em toda sua plenitude. O exemplo vai te ajudar na execução de sua tarefa.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Roteiro de prática

Implementando Interfaces aninhadas

Primeiramente, é importante responder à pergunta que formulamos: existe relação de herança em interfaces aninhadas? A resposta é não! Quando aninhamos uma interface ou classe, elas se tornam membro da entidade mais externa, não herdando qualquer de seus métodos ou variáveis.

Mas então o que ocorre no código 29? Como a classe “Concreta” implementa o método “getNome()” da interface “iExterna”? Antes de respondermos a isso, altere a linha 4 do código 30, modificando a assinatura do método. Você pode mudar o nome, por exemplo. Execute o código novamente e veja se houve alguma alteração na saída.

Essa pequena mudança explicita o que está acontecendo. Na verdade, a classe “Concreta” nunca implementou o método “getNome()” da interface “iExterna”, o que ocorreu foi apenas uma coincidência de nomes. Caso contrário, ao alterar a assinatura do método em “iExterna” o compilador teria acusado que o método não estava sendo implementado. Vamos modificar mais um pouco nosso código: modifique a classe “Concreta” para que ela agora implemente a interface “iExterna”. Se você tentar compilar o código, verá que será acusado um erro, uma vez que “Concreta” não implementa o método abstrato de “iExterna”. Agora, vamos alterar “iExterna” como mostrado a seguir:

JAVA

Código 31: Interface iExterna modificada.

Na classe “Concreta”, iguale a assinatura do método da linha 6 (código 29) ao método abstrato de “iExterna” e implemente o método “getNomeInterna()” de “iInterna” como mostrado no código 32. Em seguida, execute o código. E então? O que aconteceu?

JAVA

Código 32: Código modificado de "Concreta" e "Principal".

Eis a saída.

JAVA

Saída após a execução do código.

Os exercícios que fizemos até agora nos permitem tirar algumas conclusões: primeiro, como já falamos, não há herança em classes ou interfaces aninhadas. Os membros da entidade mais externa não são herdados pelas internas. Em segundo, em um aninhamento de interfaces, uma classe só tem acesso aos métodos da classe que implementa. Por isso, no nosso exemplo anterior, mesmo sendo “iInterna” uma interface membro de “iExterna”, seus métodos não são acessíveis para implementação por “Concreta”, o que explica a saída vista anteriormente. O mesmo vale caso a classe implemente uma interface interna. Os métodos da interface externa também não podem ser implementados por essa classe.

Para finalizar, vamos criar a classe “Base” com a interface membro “iNested”, como visto no código 33. Agora, altere o código 32 para que “Concreta” implemente “iNested”.

JAVA

Código 33: Classe Base.

Se você tentou simplesmente alterar a linha 4 para ela implementar “iNested” fazendo “class Concreta implements Base.iNested”, verá que isso não dará certo. Isso ocorre, porque “iNested” é membro da classe ”Base”. Assim, para que “iNested” esteja disponível em concreta, basta que “Concreta” estenda “Base”. Ao fazer isso, ela herdará a interface membro “iNested”, que agora pode ser implementada como vemos em código 34.

JAVA

Código 34: Implementando a Interface iNested.

A saída será:

JAVA

Código 34: Implementando a Interface iNested.

Faça você mesmo!

Considere os trechos de código a seguir:

JAVA

Código 35: Classe IdUnico.

JAVA

Código 36: Classes Concreta e Principal.

Ao tentar compilá-lo, o compilador informou que havia erro. Sabendo que o problema está nos trechos código mostrados, assinale a única opção que apresenta a solução correta para sanar o problema e permitir que o programa seja compilado e funcione como previsto.

A É suficiente alterar o modificador de acesso da variável “seed” para “protected”.

B O modificador de acesso da interface “iCodigo” precisa ser “public”.

C O modificador de acesso da interface “iCodigo” precisa ser “default” (omitir o modificador).

D O modificador de acesso da interface “iCodigo” precisa ser “protected”.

E É suficiente que o modificador da variável “seed” e da Interface “iCodigo” sejam “protected”.

Parabéns! A alternativa A está correta.

Essa é uma situação interessante. Ambas as classes são do mesmo pacote e possuem hierarquia de herança. Assim, para estar acessível em “Concreta”, a variável “seed” precisa ser, no mínimo, “protected” (protegida). A interface “iCodigo” é um membro privado, entretanto, esse modificador aplica-se somente a ela. O método “imprimeCod ()” é público, sendo, assim, possível implementá-lo em “Concreta”.

O que você aprendeu neste conteúdo?

Este estudo contou com um conteúdo extenso, mas no qual abordamos vários pontos importantes. Vamos recapitular os principais:

1. Classes e métodos abstratos.
2. Métodos e classes Final.
3. Atribuições permitidas entre variáveis da superclasse e subclasse.
4. Polimorfismo.

5. Interfaces – conceito e uso.
6. Aninhamento de interfaces e classes.
7. Herança e interfaces aninhadas.
8. Programação funcional e interface funcional.
9. Emprego avançado de Interfaces.

Explore +

A programação funcional é um paradigma suportado por outras linguagens, além da linguagem Java. Conhecê-la certamente ampliará o seu arcabouço técnico, contribuindo para que você se destaque. A Java oferece extenso suporte a esse tipo de programação e o usa em sua API. Você pode pesquisar sobre o assunto na internet. Há bastante informação. Mas não deixe de ver, também, a documentação oficial da linguagem no site da Oracle. Aliás, esse é um bom ponto de partida.

Também no site da Oracle, na página dos aplicativos Apex, conheça maiores detalhes sobre o Lambda.

Acesse também o blog dedicado às funcionalidades Java, o Java Magazine, e leia o artigo **Functional programming in Java: Lambda reuse and lexical scoping**, de Venkat Subramaniam, para descobrir como usar expressões lambda para reduzir significativamente a confusão de código.

Referências

GEEKSFORGEEKS. **Top 10 Programming Languages to Learn in 2023**. geeksforgeeks. Consultado na internet em: 6 maio 2023.

LISKOV, B. H.; WING, J. M. **A Behavioral Notion of Subtyping**. ACM Transactions on Programming Languages and Systems, v. 16, n. 6, p. 1811–1841, Nov 1994. Consultado na internet em: 06 set 2023.

ORACLE AMERICA INC. **Java 10 Local Variable Type Inference**. Consultado na internet em: 17 fev. 2021.

ORACLE AMERICA INC. **Lesson**: Introduction to Collections (The JavaTM Tutorials > Collections). Consultado na internet em: 22 jan. 2021.

ORACLE AMERICA INC. **Object (Java SE 15 & JDK 15)**. Consultado na internet em: 20 fev. 2021.

ORACLE AMERICA INC. **Primitive Data Types (The JavaTM Tutorials > Learning the Java Language > Language Basics)**. Consultado na internet em: 17 fev. 2021.

ORACLE INC. **Object (Java SE 17 & JDK 17)**. Consultado na internet em: 13 maio 2023.

ORACLE INC. **TreeSet (Java Platform SE 8)**. Consultado na internet em: 13 maio 2023.

TIOBE. **TIOBE Index**. Consultado na internet em: 6 maio 2023.

Material para download

Clique no botão abaixo para fazer o download do conteúdo completo em formato PDF.

Download material

O que você achou do conteúdo?

☆ ☆ ☆ ☆ ☆

Relatar problema